CSE311 ARTIFICIAL INTELLIGENCE

Mini Project Report

SAKSHI

Multi-Camera Vehicle Analytics and Evidence Review

An AI system for vehicle detection, licence-plate review and auditable observations

Submitted by

Student name	Roll number
A S Sri Dathan	2024BCS0091
Akshay V.S	2024BCD0015
Bharadwaj B Chand	2024BCS0059
Shivansh Neekhra	2024BCS0023
Srimoneyshankar Ajith	2024BCS0055

Under the guidance of
Dr. Sreeja M U

Department of Computer Science and Engineering
Indian Institute of Information Technology Kottayam

Third Year • Academic Year 2026–2027

Acknowledgement and Declaration

We express our sincere gratitude to our course instructor, Dr. Sreeja M U, for guidance throughout this mini project, and to the Department of Computer Science and Engineering, IIIT Kottayam, for the facilities provided.

We declare that this report describes work carried out by our team. Third-party resources are acknowledged below and in the References. The project uses pretrained detection and recognition models and demonstration footage; it does not claim that the team trained the detector or the OCR model from scratch. Each member has reviewed, run and understood the code, methodology, data and results reported here and is responsible for them.

Third-party resources used

- Ultralytics YOLOv8n pretrained weights (yolov8n.pt) for vehicle and person detection.
- Fast-ALPR 0.4.0 with the Open Image Models plate detector (YOLOv9-tiny, 384 × 384) and the FastPlateOCR global MobileViT-v2 OCR model.
- PyTorch (CPU), ONNX Runtime, OpenCV, NumPy, FastAPI, SQLite, React, TypeScript, Vite and Tailwind CSS.
- Six short recorded traffic clips (CAM-01 to CAM-06) for local demonstration and benchmarks; CAM-07 to CAM-09 were supplied privately for plate-review experiments and are not redistributed.

Table of Contents

Abstract vi
1 Introduction 1
1.1 Background 1
1.2 Motivation 1
1.3 Why artificial intelligence is relevant 1
1.4 Project scope 1
1.5 Report organisation 1
2 Problem Statement and Objectives 2
2.1 Problem statement 2
2.2 Objectives 2
2.3 Scope and measurable objectives 2
2.4 Success criteria 2
3 Literature Review 3
3.1 Object detection 3
3.2 Tracking by detection 3
3.3 Licence-plate detection and text recognition 3
3.4 Efficient inference and evaluation 3
3.5 Research gap addressed 3
3.6 Implications for the design 4
4 Proposed System and Methodology 5
4.1 System overview 5
4.2 Inputs and frame preparation 5
4.3 Vehicle and person detection 6
4.4 Camera-local tracking 6
4.5 Plate recognition and abstention 6
4.6 Time and cross-camera journeys 7
4.7 Data and evidence flow 7
5 System Implementation 8
5.1 Software components 8
5.2 Libraries and model assets 8
5.3 Query and review behaviour 8
5.4 Privacy and audit controls 8
6 Data and Evaluation Method 9
6.1 Available video and review inputs 9
6.2 Dataset preparation and leakage controls 9
6.3 Evaluation protocol 9
6.4 Operational metrics 9
6.5 Interpreting future evaluation results 10
7 Results and Discussion 11
7.1 Nine-camera replay profile 11
7.2 Detector runtime comparison 12
7.3 Selected plate-crop diagnostic review 13
7.4 What the results support 15

7.5 Discussion 15
7.6 Projected scaling of the architecture 15
8 Limitations and Ethical Considerations 18
8.1 Responsible interpretation 18
9 Future Scope and Conclusion 19
9.1 Future work 19
9.2 Conclusion 19
References 20
Appendix A: Reproducibility and Viva Preparation 21
A.1 Running the system 21
A.2 Questions the team should be ready to answer 21

List of Figures

Figure 4.1 SAKSHI system architecture and observation flow 5
Figure 4.2 Plate OCR model: convolutional stem, MobileViT-v2 transformer-style blocks and character head 6
Figure 4.3 Plate-reading decision path and abstention behaviour 7
Figure 7.1 Sampled frames completed and attempts over the 500 ms target, by camera 12
Figure 7.2 YOLOv8n median model-forward latency by runtime 13
Figure 7.3 Selected plate-crop diagnostic: reviewer judgement and gate outcomes 14
Figure 7.4 Projected aggregate sampled FPS for four deployment scenarios 16
Figure 7.5 Measured single-host deployment and projected scaled deployment 17

List of Tables

Table 2.1 Objectives and the evidence available for each 2
Table 3.1 Relevant methods and their role in SAKSHI 4
Table 5.1 Main code modules and responsibilities 8
Table 6.1 Inputs represented in the available review records 9
Table 6.2 Evidence required before making model-quality claims 10
Table 7.1 Five-minute active-inference profile (local 12-logical-CPU Linux host) 11
Table 7.2 Detector-only CPU benchmark 13
Table 7.3 Summary and limits of the selected crop diagnostics 14
Table 7.4 Supported findings and claims still open 15
Table 7.5 Projected capacity at one sampled frame per second per camera 16
Table 9.1 Viva preparation: questions and where the answer is in this report 21

Abstract

SAKSHI is a multi-camera vehicle analytics system that converts recorded traffic footage into reviewable, auditable observations. Vehicles and people are detected with a pretrained YOLOv8n network, and a two-stage licence-plate path (a plate detector followed by a MobileViT-v2 optical character recognition model) proposes plate text. A single bounded scheduler samples all cameras under a global frame budget, and a camera-local motion-and-box tracker associates detections over time. Plate readings pass through normalisation, confidence and format checks, and a three-read per-track consensus, so uncertain or conflicting readings never become an identity. A FastAPI backend stores observations and evidence metadata in SQLite, and a React and TypeScript dashboard supports monitoring, deterministic queries, local rules and hash-verifiable evidence export.

In the latest nine-camera replay, 349 sampled frames were processed in 298.011 seconds (1.171 frames per second) with zero failed inference attempts. A detector-only benchmark on 60 calibration frames measured a median model-forward latency of 92.834 ms for PyTorch FP32 and 55.782 ms for static INT8 ONNX. Because no independent held-out labelled dataset was available, these are execution measurements only and no claim is made about detector precision or plate accuracy. The report also projects how the same architecture would scale on stronger hardware, clearly marked as an estimate, since the full target deployment could not be tested within the available hardware and budget.

Keywords: vehicle detection, licence-plate recognition, OCR, MobileViT, multi-object tracking, multi-camera video, evidence integrity

1 Introduction

1.1 Background

Fixed cameras collect long video streams, but an investigator usually needs only a few observations: a vehicle class, a possible plate reading, a camera and an approximate time. Searching every recording by hand takes time and makes repeated review difficult. Computer vision can reduce footage to candidate observations that a person can inspect, while a data service preserves how each observation was produced.

1.2 Motivation

A detector can find a car in a frame, but operational questions concern repeated observations across time. Plate OCR can misread a small or blurred registration, and a replay timestamp can be mistaken for the original capture time. SAKSHI addresses this wider workflow by retaining raw reads, recording model provenance, marking uncertain identities, and keeping measured system behaviour separate from unmeasured recognition accuracy.

1.3 Why artificial intelligence is relevant

Learned visual models are used where hand-written rules are weak: object detection identifies vehicles in varied scenes, and OCR converts a plate crop into a character sequence. Tracking and consensus then organise the repeated predictions. These later stages are deterministic algorithms rather than newly trained models. The AI contribution is therefore pretrained detection and recognition integrated with rule-based association, validation and review.

1.4 Project scope

The current implementation processes recorded sources and can be configured for RTSP camera streams. Demonstrated evidence is local replay and software behaviour. The system does not establish city-wide vehicle identity, calibrated speed, congestion estimates or production surveillance readiness. These boundaries are treated as design and evaluation constraints throughout the report.

1.5 Report organisation

Section 2 defines the problem and objectives. Section 3 reviews detection, tracking, OCR and quantisation work. Sections 4 and 5 describe the methodology and implementation. Sections 6 and 7 document the data, evaluation method and results, including a projection of how the architecture scales. Sections 8 and 9 discuss limitations, future work and conclusions.

2 Problem Statement and Objectives

2.1 Problem statement

Given video from several fixed cameras, how can a system detect vehicles, propose plate readings, preserve the history and provenance of each observation, and help an investigator review candidate activity without turning a low-quality prediction into an unsupported identity?

2.2 Objectives

- Run one shared background processor that samples available cameras under a configurable global frame budget.
- Use pretrained computer-vision models to detect vehicles and, when enabled, people for local privacy masking.
- Detect and read plate regions, normalise candidate text, and retain raw, invalid, low-confidence and ambiguous observations for review.
- Associate detections within a camera using motion and bounding-box cues, retiring tracks when assignment is ambiguous.
- Persist observations, camera metadata, model hashes, image derivatives, review actions and audit records.
- Provide a dashboard for camera monitoring, bounded queries, local rules, analytics and evidence exports.
- Provide reproducible data-manifest and evaluation tools, and report what they can and cannot measure.

2.3 Scope and measurable objectives

Table 2.1 Objectives and the evidence available for each

Objective	Evidence available	Open criterion
Shared multi-camera processing	Recorded nine-source 300-second replay profile.	Sustained target-device rate and live stream behaviour.
Vehicle and plate AI	Pretrained models load and run; a small review audit exists.	Held-out detector and exact-plate metrics.
Tracking	Deterministic test scenarios exercise association and ambiguity rules.	Field ID-switch and fragmentation rates.
Evidence handling	Hash references, immutable snapshots and optional signing are implemented.	Independent provenance and operational key controls.
Cross-camera journey	Clock and route constraints are represented in code.	Verified clocks, surveyed locations and labelled route pairs.

2.4 Success criteria

For this coursework, success means that the system demonstrates a meaningful AI application, keeps uncertainty visible, produces reproducible software measurements, and makes no claim stronger than its labels support. An accuracy claim requires a complete labelled test split that is independent by vehicle, camera, location and session.

3 Literature Review

SAKSHI combines established methods rather than proposing a new detector or recognition architecture. The literature informs the choice of compact visual models, association algorithms and evaluation discipline. Table 3.1 relates each work to the implementation.

3.1 Object detection

Redmon et al. framed detection as a single network that predicts bounding boxes and class probabilities directly from an image, establishing the You Only Look Once family of real-time detectors [1]. SAKSHI uses the Ultralytics YOLOv8 nano checkpoint as a pretrained detector for vehicle classes and, in privacy mode, people [2]. The project's CPU benchmark measures execution only; no labelled result shows how well the checkpoint detects vehicles in these particular clips.

3.2 Tracking by detection

SORT uses motion estimates and the Hungarian assignment algorithm to link detections efficiently [3]. DeepSORT adds a learned appearance metric to maintain identities through occlusion [4]. SAKSHI instead uses a camera-local motion-and-box cost with global assignment and an explicit ambiguity margin. It has no learned appearance embedding, so its track identifiers are local associations rather than persistent vehicle identities.

3.3 Licence-plate detection and text recognition

Laroca et al. describe an automatic plate-recognition pipeline built on YOLO detection followed by character recognition [5]. Fast-ALPR exposes a plate detector followed by an OCR stage [6]. Scene-text research such as CRNN treats recognition as transcription of an image sequence without first segmenting every character [7]. The selected global plate OCR asset is documented as a MobileViT-v2 model [8, 9], a hybrid of convolution and transformer-style self-attention designed for mobile hardware. Its upstream training data and vendor-reported results are not measurements of this project and are not proof of Indian-plate accuracy.

3.4 Efficient inference and evaluation

Integer quantisation can reduce model size and inference time using lower-precision arithmetic, but the result depends on calibration and model behaviour [10]. SAKSHI benchmarks a static INT8 export against FP32 on the same unlabelled sample frames, reporting speed and output agreement rather than accuracy. The evaluation tools also report rate intervals using the Wilson approach [11], although confidence intervals cannot repair missing or unrepresentative labels.

3.5 Research gap addressed

The project contribution is an integrated review workflow around uncertain predictions: separate raw and supported plate readings, model hashes, persistent observation histories, audit records, a query interface and evidence exports. These engineering safeguards do not make the perception models accurate. They make it easier to inspect how a result was produced and to decline an identity claim when evidence is weak.

Table 3.1 Relevant methods and their role in SAKSHI

Work or method	Main idea	Relationship to SAKSHI
YOLO [1, 2]	Single-stage detection of image regions and classes.	Pretrained YOLOv8n provides vehicle and person detections; local accuracy is unmeasured.
SORT [3]	Motion-based online association with global assignment.	SAKSHI uses assignment with custom motion, geometry and ambiguity rules.
DeepSORT [4]	Adds learned appearance association to SORT.	Comparison only; no appearance embedding or cross-camera re-identification.
YOLO-based ALPR [5]	Detect plates, then recognise characters.	Supports the modular plate detector plus OCR structure.
CRNN [7]	End-to-end sequence recognition for scene text.	OCR background; not the selected architecture.
MobileViT v2 [8, 9]	Lightweight transformer-style vision family with separable self-attention.	Global pretrained OCR component; Indian-scene validation remains open.
Integer quantisation [10]	Lower precision trades size and latency against output behaviour.	Motivates the INT8 comparison; no accuracy claim follows.

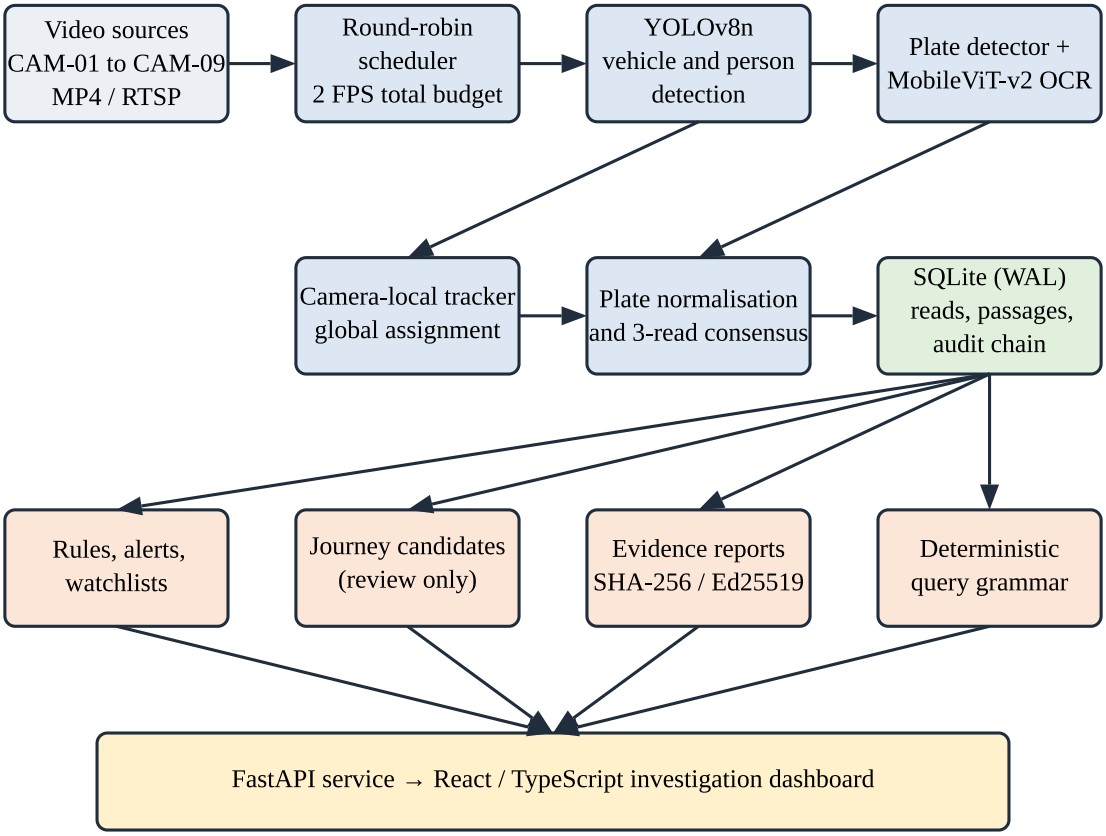
3.6 Implications for the design

A detector proposes boxes, a tracker associates them within one camera, and OCR proposes plate text; each stage has different failure modes and needs its own evidence. SAKSHI therefore keeps plate and vehicle tracks separate, preserves rejected readings for review, and avoids treating track assignment as a learned identity. The literature also highlights a gap the project has not closed: representative, independently labelled evaluation. Before deployment or a model comparison, held-out data reflecting the real cameras and plate conditions would be needed, with uncertainty and abstention reported alongside recognition metrics.

4 Proposed System and Methodology

4.1 System overview

The system has six stages: camera-source resolution, shared sampling, AI inference, camera-local association with plate review gates, persistent storage, and operator-facing services. A single processor owns inference. Browser requests read cached frames and API results, so adding viewers does not create more model workers. Figure 4.1 shows the data flow.



Authors' diagram of the implemented pipeline.

Figure 4.1 SAKSHI system architecture and observation flow

4.2 Inputs and frame preparation

Inputs can be recorded MP4 sources or configured RTSP sources. Source management distinguishes original footage from browser-compatible playback copies, records file digests and validates selected media before activation. The processor decodes sampled frames and tracks replay position separately from processing time. The vehicle model receives a 640 × 640 input; the plate detector uses its configured 384-pixel input.

The default scheduler has a 2-frame-per-second total budget, not 2 frames per second per camera. It cycles across available sources, and missing or retrying sources do not consume inference slots. Recorded video playback is separate from the low-rate sampled detection view.

4.3 Vehicle and person detection

The backend loads the YOLOv8n checkpoint lazily and runs it on CPU in the documented evaluation configuration. The detector filters for the COCO car, motorcycle, bus and truck classes. Person boxes are requested when privacy masking is enabled, and local pixelation is applied before sampled output is shared. Confidence thresholds control candidate generation, but model confidence is not a calibrated probability and cannot be treated as accuracy.

4.4 Camera-local tracking

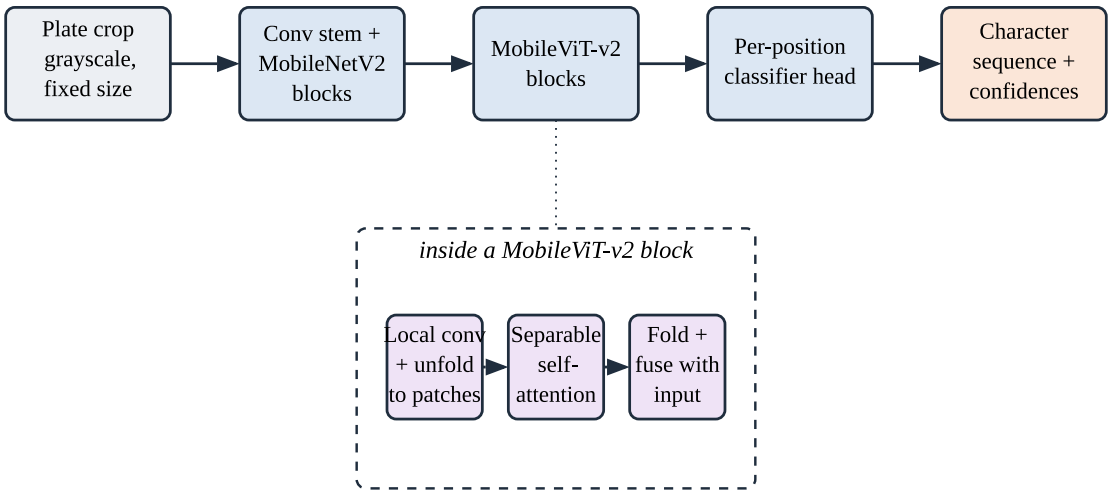
Each camera keeps its own active tracks. For a candidate track and detection, the cost combines predicted centre displacement, bounding-box overlap and change in box area. Class mismatch, implausible size change and distance outside a gate reject a pair. The tracker uses:

$$cost = 0.75 \times normalised\ distance + 0.20 \times (1 - IoU) + 0.05 \times normalised\ area\ change$$

A minimum-cost assignment with an unmatched option is solved per frame. Near-equal competing matches are marked ambiguous and the implicated older tracks are retired. These costs are heuristic association scores, not identity probabilities. Track identifiers reset with camera sessions, and no appearance model links vehicles across cameras.

4.5 Plate recognition and abstention

The plate branch runs a YOLOv9-tiny 384 × 384 plate detector and the FastPlateOCR global MobileViT-v2 OCR model through ONNX Runtime. Figure 4.2 sketches the OCR network: a convolutional stem and MobileNetV2-style blocks extract local features, MobileViT-v2 blocks unfold feature maps into patches and apply separable self-attention to model global layout, and a per-position head outputs the character sequence with confidences. Separable self-attention has linear rather than quadratic cost in the number of patches, which is why this family suits CPU inference.



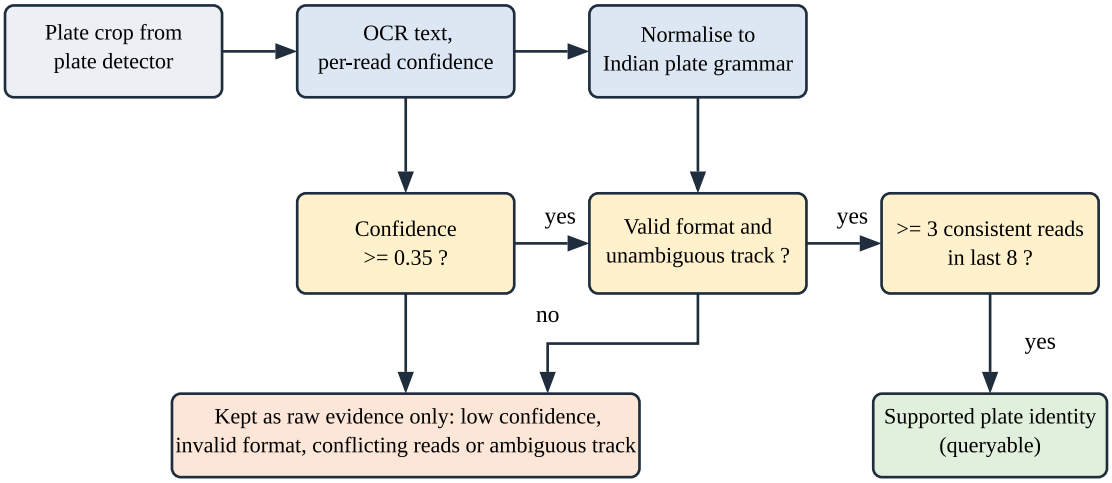
Authors' sketch based on the MobileViT and MobileViT-v2 papers [8, 9]; layer sizes omitted.

Figure 4.2 Plate OCR model: convolutional stem, MobileViT-v2 transformer-style blocks and character head

The source code normalises candidate strings to supported Indian registration syntax and records the original OCR text, canonical value, confidence and normalisation actions. Plate tracks are separate from vehicle tracks; no validated association between them is claimed.

A candidate below the 0.35 OCR confidence threshold, outside the accepted format, or associated with an ambiguous track remains in observation history and is excluded from supported plate queries. For

eligible reads, consensus is scoped to camera and track, keeps up to eight recent readings, and supports a plate only after at least three consistent readings. Conflicting strings remain conflicting, and character-level suggestions are review-only (Figure 4.3).



Authors' diagram of backend/inference.py and backend/fusion.py behaviour.

Figure 4.3 Plate-reading decision path and abstention behaviour

4.6 Time and cross-camera journeys

An observation can carry replay position, processing completion time and, when an operator supplies it, an original-time claim marked unverified. Replay position is not the time when a real-world camera recorded the scene. The runtime does not emit a synchronised capture clock, so physical travel-time calculations across cameras remain unresolved. Candidate journey links require route and time constraints and still represent associations for review, not proof of identity.

4.7 Data and evidence flow

The processor stores frame-level reads and camera-local passage summaries in SQLite with a write-ahead log. Observations record source, session, frame position, and model and pipeline-component digests. The system attempts SHA-256-referenced JPEG derivatives for sampled reads and records missing image coverage when capture or storage fails. Evidence reports snapshot the selected interval and supporting reads. A hash can detect a change to stored report bytes, and an optional Ed25519 signature can verify a report against an independently trusted public key. Neither proves that a scene or OCR result is true.

5 System Implementation

5.1 Software components

Inference ownership is separate from interface activity. The backend coordinates sources and models, validates API requests, stores records and applies review rules. The web client requests data and cached images. Evaluation commands operate on explicit records and manifests instead of silently treating demonstration footage as ground truth.

Table 5.1 Main code modules and responsibilities

Component	Module path	Purpose
Inference	backend/inference.py	Loads YOLO and Fast-ALPR, runs visual models, records plate confidence.
Scheduling and sources	backend/processing.py, backend/cameras.py	Round-robin sampling, decode state, health counters, source availability.
Tracking and fusion	backend/tracking.py, backend/fusion.py	Within-camera association and per-track OCR consensus.
Persistence and API	backend/events.py, backend/main.py	Stores observations and rules; camera, query, analytics and report endpoints.
Web dashboard	frontend/	React and TypeScript camera wall, investigations, analytics, alerts and evidence views.
Evaluation tools	evaluation/	Manifest checks, labelled-record metrics, ablations, runtime profiling, frame-review preparation.

5.2 Libraries and model assets

The model card records Python 3.12, PyTorch 2.14 (CPU), Ultralytics 8.4.155, ONNX Runtime 1.30.0, Fast-ALPR 0.4.0, FastPlateOCR 1.1.0 and Open Image Models 0.6.0. The wider application uses OpenCV and NumPy for video and image operations, FastAPI for HTTP services, SQLite for local persistence, and React, TypeScript and Vite for the browser interface.

The vehicle checkpoint and the plate detector and OCR assets are identified by cryptographic digests in the model bundle. Provenance identifies the exact files used; it does not establish training data, model rights, validation quality or deployment approval. Model-weight terms should be checked independently before redistribution.

5.3 Query and review behaviour

Queries use a bounded grammar for plate, camera, class and time, and unsupported questions are rejected explicitly. The system has no language model, no generated SQL and no external registration lookup. Local watchlists and zone rules create review alerts; a zone alert does not prove unauthorised access.

5.4 Privacy and audit controls

Privacy mode pixelates sampled people and plates and disables raw playback. Optional authentication adds roles, camera and case scopes, CSRF protection, TOTP multi-factor login and access audits. Hash-chained audits and optional Ed25519 signatures help detect changes to checkpoints and evidence reports. Independent storage, key management and source authenticity remain necessary, so this is a local coursework system and not a security certification.

6 Data and Evaluation Method

6.1 Available video and review inputs

Six bundled playback clips, CAM-01 through CAM-06, are recorded at 960 × 540 and a reported 25 frames per second. Three further managed original sources, CAM-07 through CAM-09, are represented in local validation records at 1920 × 1080. Camera file names do not establish where or when footage was captured.

Table 6.1 Inputs represented in the available review records

Sources	Role in this report	Known limits
CAM-01 to CAM-06	Bundled playback clips for local replay and detector calibration.	Not a labelled test set; location, session independence and capture time unverified.
CAM-07 to CAM-09	Managed originals for selected plate reviews and active replay.	Origin, authorisation, timezone and camera metadata not independently verified.
45-frame review set	Five prediction-blind samples per camera from equal-duration strata.	Prepared 18 October 2026; no human labels prefilled; not held out.
21 selected plate crops	Small human readability review.	Selected private examples; 16 unreadable; not representative.

6.2 Dataset preparation and leakage controls

The review-set generator hashes source clips, selects frame positions without inspecting predictions, and records image hashes. The 45-frame set contains five frames from each of nine sources and, at preparation time, no human vehicle counts or plate text. The evaluation manifest requires clip hashes, explicit labels, train, validation and test membership, and separation by vehicle, camera, location and session. Repeated vehicles or adjacent frames must not be treated as independent test examples. The 45-frame set is a labelling workflow, not proof that a model meets an accuracy target.

6.3 Evaluation protocol

When labels are ready, the detector evaluator matches same-class predicted and reference boxes one-to-one at an intersection-over-union threshold of 0.50 and reports precision and recall with denominators. OCR evaluation compares canonical strings exactly and reports correct, incorrect, missing and abstained outputs separately. Tracking evaluation requires camera, session and chronological frame labels, and counts identity switches and visible-track fragmentation. Journey evaluation needs independently validated locations and synchronised clocks. Paired ablations require identical held-out samples and exact model provenance across variants.

6.4 Operational metrics

Runtime profiling reports sampled frames per second, processing failures, phase timings, endpoint latency, CPU and resident memory. Model confidence is shown separately from measured accuracy. These measures describe software behaviour under recorded conditions; they do not measure the fraction of vehicles or plates recognised correctly.

Table 6.2 Evidence required before making model-quality claims

Claim	Required ground truth	Metric reported by project tools
Vehicle detection	Complete class-labelled boxes on held-out frames.	One-to-one class-aware precision and recall at IoU 0.50.
Plate OCR	Reviewed readable plate text with matched crop regions.	Exact string match, coverage, errors and abstentions.
Tracking	Vehicle identities across ordered frames in independent sessions.	ID switches, fragmentation, matched identity counts.
Cross-camera association	Verified locations, synchronised clocks, labelled vehicle pairs.	Link precision and recall, false-candidate rates with uncertainty.
Runtime	Host, model, branch settings and sample window.	Latency, throughput, failures, CPU and memory.

6.5 Interpreting future evaluation results

A fair comparison reports model quality and coverage together. For OCR, exact-string accuracy on readable crops should be paired with a coverage or abstention rate, with unreadable and uncertain labels shown separately; otherwise a conservative gate can look accurate simply by abstaining on hard examples. Every result should include sample counts, exclusions, model hashes and environment details.

7 Results and Discussion

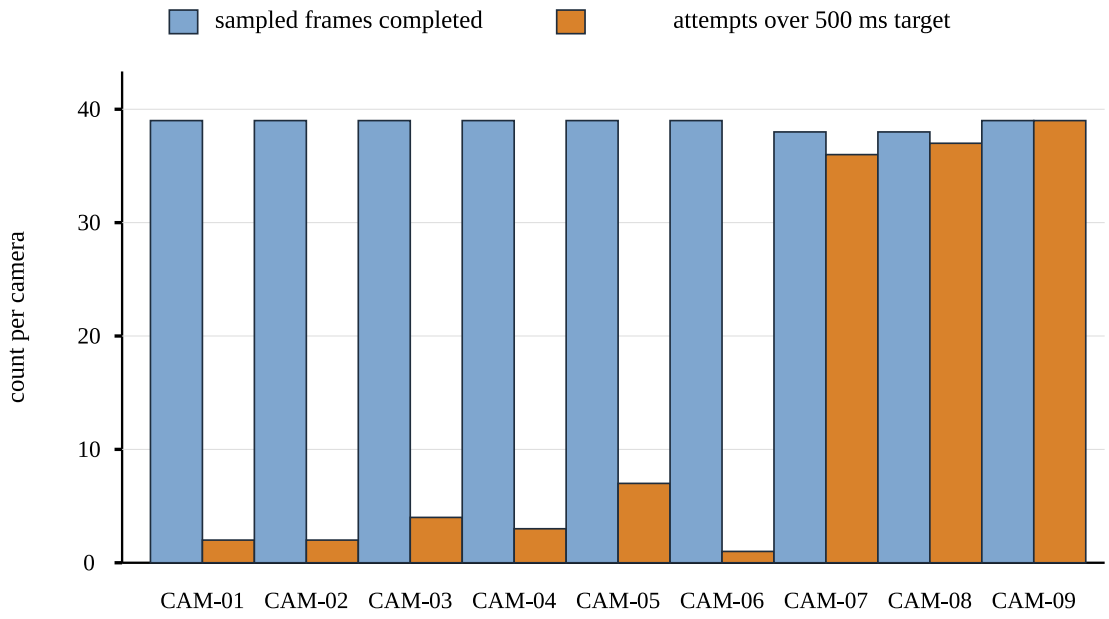
Available results measure local runtime, model-forward latency and a small crop-review sample. They do not establish general recognition accuracy. Section 7.6 adds a clearly labelled projection of how the architecture would scale.

7.1 Nine-camera replay profile

A five-minute local replay enabled vehicle detection and plate recognition for all nine sources. The valid counter window lasted 298.011 seconds. The processor completed 349 sampled frames, or 1.1711 frames per second against the configured 2.0 total budget. No inference attempts or health reads failed. Completion counts were balanced at 38 or 39 per camera, but the measured system did not reach its configured rate.

Table 7.1 Five-minute active-inference profile (local 12-logical-CPU Linux host)

Measure	Observed result	Interpretation
Available sources	9 of 9 processing at start and end	Replay evidence for all configured sources.
Valid counter window	298.011 s	No invalid sample gap found.
Completed samples	349; 1.1711 FPS total	Below the configured 2.0 FPS total budget.
Failed attempts	0; health-read failures 0	No failures in this bounded window.
CPU usage	163.5% median; 226.9% p95	Process CPU can exceed 100% across cores.
Resident memory	830.2 MiB median; 851.5 MiB p95	Local measurement, not a device requirement.
Plate-source overruns	CAM-07 36/38; CAM-08 37/38; CAM-09 39/39	Attempts exceeded the 500 ms target; not dropped frames.



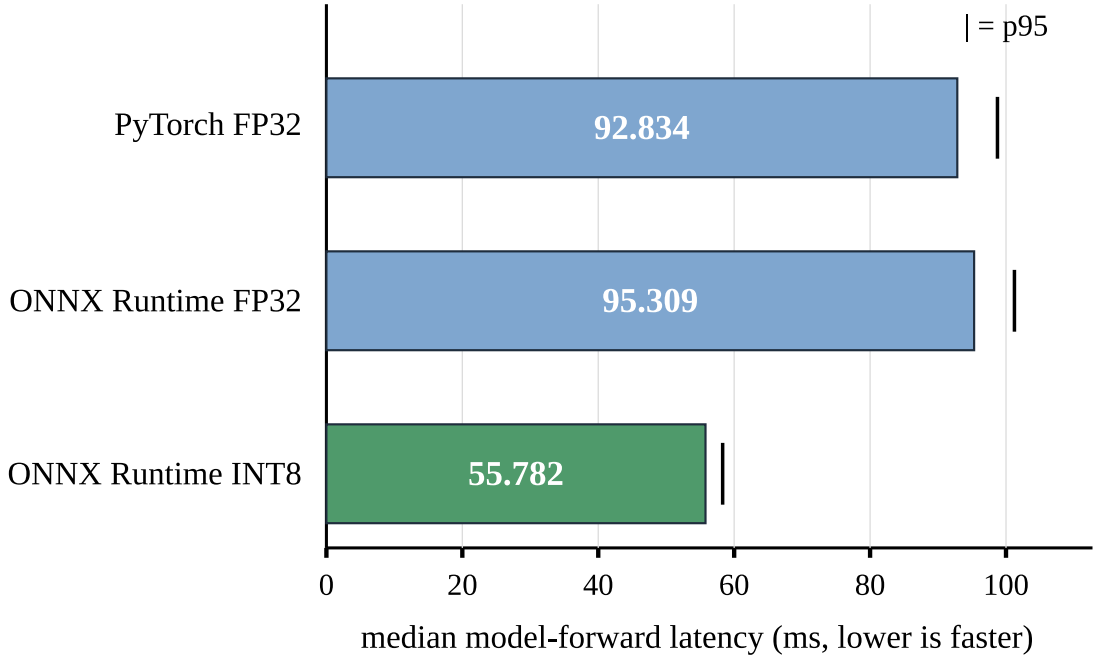
Validation profile of 18 October 2026. Counts are sampled outputs, not every decoded source frame.

Figure 7.1 Sampled frames completed and attempts over the 500 ms target, by camera

The high-resolution plate cameras (CAM-07 to CAM-09) account for nearly all budget overruns, which indicates that plate inference on 1920 × 1080 sources, not vehicle detection, is the main throughput bottleneck on this host.

7.2 Detector runtime comparison

The benchmark used sixty sampled frames from the six bundled clips to calibrate a static quantised export. It measured ten model-forward runs after two warm-ups on a 12th-generation Intel Core i5-12450H CPU. The input was 640 × 640; decoding, preprocessing, non-maximum suppression, rendering, database writes and OCR were outside the measurement.



Detector-only CPU benchmark of 14 October 2026; not end-to-end pipeline time.

Figure 7.2 YOLOv8n median model-forward latency by runtime

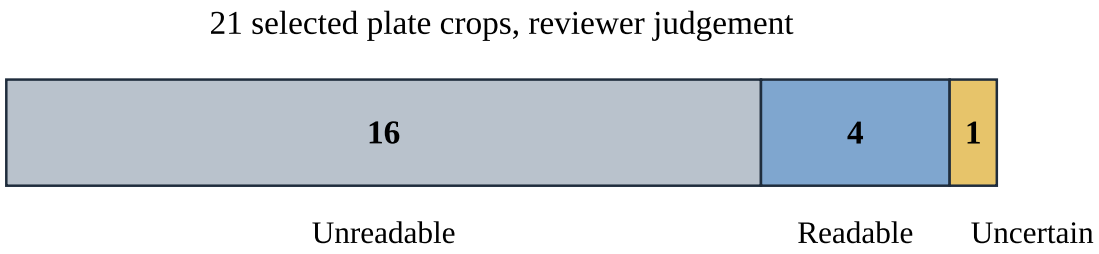
Table 7.2 Detector-only CPU benchmark

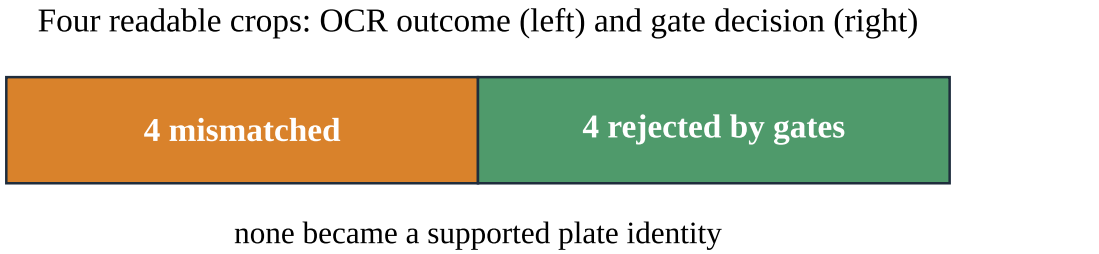
Runtime	Median / p95 latency	Model file size	Sample scope
PyTorch FP32	92.834 / 98.743 ms	6.55 MB	10 measured runs
ONNX Runtime FP32	95.309 / 101.240 ms	12.82 MB	10 measured runs
ONNX Runtime INT8	55.782 / 58.306 ms	3.60 MB	10 measured runs; static QDQ

INT8 reduced median model-forward latency by about 41% relative to FP32 ONNX and reduced the exported file from 12.82 MB to 3.60 MB. On the same sixty unlabelled calibration frames, INT8 matched 15 of 16 FP32 reference detections under class-aware IoU matching at 0.50. With no ground-truth labels, this is model-to-model agreement, not precision, recall or proof that the INT8 export is accurate. The quantised artefact is not yet integrated into live inference.

7.3 Selected plate-crop diagnostic review

This is a selected diagnostic sample, not a representative test set. A reviewer assessed 21 crops from CAM-07 through CAM-09: 16 were unreadable, 4 readable and 1 uncertain. Only the 4 readable crops support direct string comparison. In the recorded pipeline results all four OCR strings differed from the reviewer transcriptions and each mismatch was rejected by the configured gates, so none became a supported plate identity. This demonstrates abstention on these examples, not OCR accuracy or a false-accept rate.





Selected sample; not representative and not an accuracy estimate.

Figure 7.3 Selected plate-crop diagnostic: reviewer judgement and gate outcomes

An offline probe applied 68 crop variants to the four readable crops. One variant matched exactly and five passed format and confidence checks (one exact, four inexact). The probe did not test track association or repeat-read consensus and was not deployed, so it is diagnostic only.

Table 7.3 Summary and limits of the selected crop diagnostics

Evidence slice	Observed result	What can be concluded
Selected crops	21 total: 16 unreadable, 4 readable, 1 uncertain.	Only 4 support direct string comparison; not representative.
Recorded pipeline	All 4 readable-crop OCR strings mismatched and were rejected; none became a supported ID.	Shows gate behaviour in these cases only; not an accuracy or false-accept rate.
Offline probe	68 variants: 1 exact match; 5 passed format and confidence checks (1 exact, 4 inexact).	Association and repeat-read gates untested; exploratory and not deployed.

7.4 What the results support

Table 7.4 Supported findings and claims still open

Supported by recorded evidence	Not established by this report
The pipeline completed a bounded nine-source replay without failed attempts.	A sustained 2 FPS rate, live-camera reliability or long-duration recovery.
The runtime shared 38 to 39 sampled outputs per camera in the window.	Full-rate analytics or equal source-frame coverage.
INT8 ran faster than FP32 ONNX in a detector-only local benchmark.	Ground-truth detector accuracy or safety of model promotion.
Plate gates rejected four mismatched outputs in a small review.	General Indian plate accuracy or a low false-identification rate.
Hashes and optional signatures can verify stored artefact integrity.	Truth of the scene, plate text or an independent chain of custody.

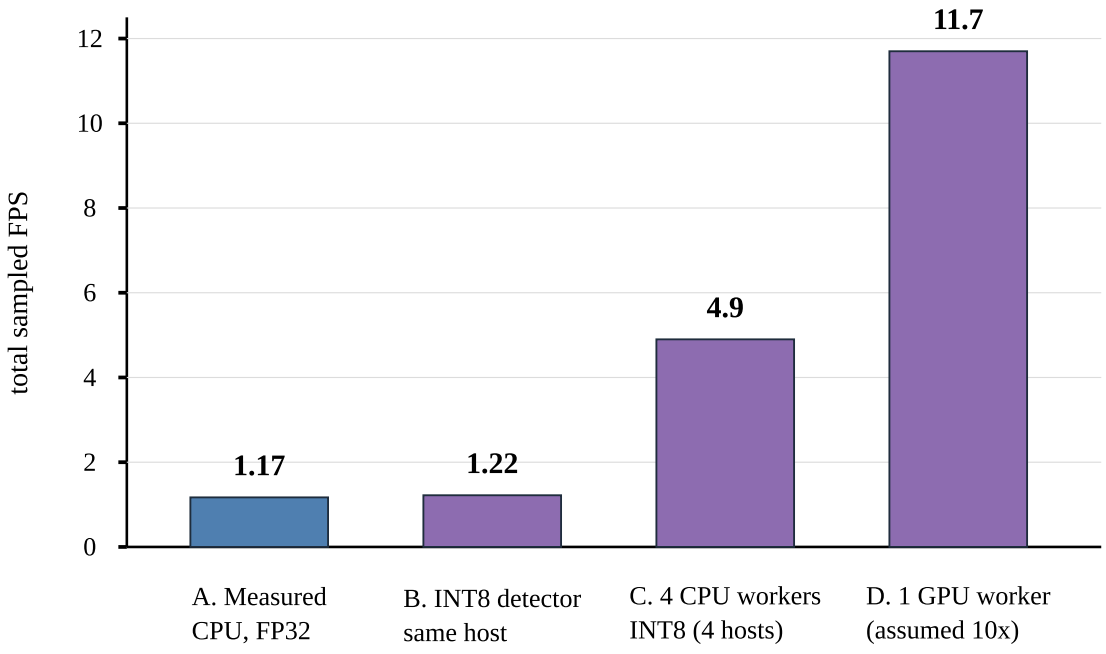
7.5 Discussion

The nine-camera replay delivered 1.17 FPS against a 2 FPS budget, and plate-camera inference often exceeded its 500 ms target. INT8 was faster and smaller than FP32 ONNX with one disagreement on unlabelled calibration frames; held-out labels are needed to judge that trade-off. In the selected crop review, all four readable-crop mismatches were rejected by the plate gates, while 16 of 21 crops were unreadable. These are workflow observations, not accuracy evidence. Taken together they show a working, failure-free pipeline whose main constraints are compute budget and plate image quality, not software stability.

7.6 Projected scaling of the architecture

The complete target deployment, with many cameras, accelerated inference and a networked database, could not be built or tested within the available hardware and budget. The architecture was nevertheless designed so that its components separate cleanly: decoding and sampling, inference, storage and presentation communicate through narrow interfaces. This subsection estimates what that separation would allow. Every figure below is a projection derived from the measurements above and stated assumptions; none was measured.

The measured host spent about 854 ms of wall-clock time per sampled frame (1 / 1.1711 FPS) with both branches enabled. Replacing the FP32 detector with the INT8 export would save roughly 37 ms of model-forward time (92.8 ms to 55.8 ms), giving about 1.22 FPS on the same host. Because the processor is stateless between frames apart from per-camera tracks, inference can be partitioned by camera across independent workers; with four such hosts and no shared bottleneck, the aggregate would be roughly four times the single-host rate. A GPU worker is assumed here to be an order of magnitude faster than the CPU path, which is an illustrative assumption rather than a benchmark.



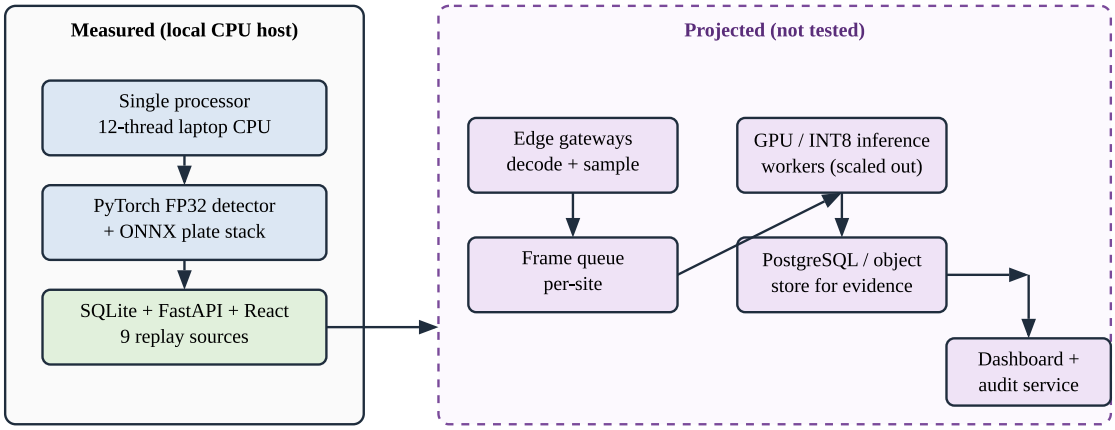
Scenario A is measured; B to D are estimates built from the stated assumptions and were not tested.

Figure 7.4 Projected aggregate sampled FPS for four deployment scenarios

Table 7.5 Projected capacity at one sampled frame per second per camera

Scenario	Basis	Aggregate FPS	Cameras served at 1 FPS each
A. Measured CPU, FP32	Nine-camera replay, both branches	1.17	about 1
B. INT8 detector, same host	Measured latency delta applied	about 1.22	about 1
C. Four CPU hosts, INT8	Linear partition by camera (assumed)	about 4.9	about 4
D. One GPU worker	Assumed 10× speed-up (illustrative)	about 11.7	about 11

These numbers show that the measured profile is a proof of concept for the logic and data flow, not a capacity limit of the design. They also show that plate recognition on full-resolution sources, which dominated the overruns, is the first component to move to accelerated hardware, and that the per-frame budget scheduler already enforces the fixed total rate that such a deployment would need. Figure 7.5 contrasts the measured setup with the projected one. Confirming any of these estimates requires the end-to-end profiling listed under future work.



Left: tested configuration. Right: projected, not tested.

Figure 7.5 Measured single-host deployment and projected scaled deployment

8 Limitations and Ethical Considerations

The evidence is strongest for implementation and bounded runtime behaviour. It is weak for model quality because labels, capture provenance and independent splits are incomplete. These limitations define what the system should not be used to claim.

- No complete held-out dataset is available. Detector precision and recall, plate exact-match accuracy, confidence calibration and field tracking rates are unknown.
- The OCR sample includes only four crops judged readable and comes from selected footage. Rejected mismatches show the abstention gate's behaviour on those examples, not a general false-identity rate.
- The 45-frame review set is prepared but unannotated. Some clips may share vehicles or locations, and source metadata does not verify capture time, timezone, camera identity or authorisation.
- Track identities are camera-local and use motion and geometry. Plate tracks are not linked to vehicle tracks by validated evidence, and there is no learned cross-camera re-identification.
- Replay timestamps and arrival times are not synchronised capture clocks, so physical speed, route travel time and city-wide unique-vehicle counts cannot be inferred.
- The latest profile is a five-minute replay on one 12-logical-CPU workstation. It does not establish live RTSP performance, target-edge suitability, multi-hour uptime, thermal behaviour or browser playback quality.
- The scaling projections in Section 7.6 are estimates, not measurements.
- Authentication and privacy masking are implemented, but the project has not undergone production security, legal or privacy assessment.

8.1 Responsible interpretation

A plate reading can identify a person's vehicle, so even an unresolved OCR candidate is sensitive. This report does not reproduce plate strings or private frame images. A practical deployment would need a documented legal basis, data minimisation, retention limits, access monitoring, secure key handling and human review before any consequential decision. A watchlist match or restricted-zone alert is a review prompt, not a finding of wrongdoing. The system should be demonstrated only with footage the team is authorised to use.

9 Future Scope and Conclusion

9.1 Future work

- Complete an authorised, prediction-blind annotation set with vehicle boxes, readable and unreadable plate labels, camera and session metadata, and source rights, keeping vehicles, sessions and locations isolated across train, validation and test splits.
- Run the evaluator on held-out detections and OCR outputs, reporting denominators, per-camera results, abstention coverage, error types and confidence intervals, without selecting thresholds on the final test split.
- Compare the pretrained OCR against a region-specific or fine-tuned model on licensed, locally representative plate data, promoting changes only after paired evaluation on identical held-out samples.
- Measure end-to-end performance on intended hardware with both branches enabled, including cold start, warm latency, throughput, skipped frames, CPU, memory, power, thermal behaviour and recovery over longer periods, and confirm or correct the projections in Section 7.6.
- Integrate the INT8 detector into live inference after held-out validation, and evaluate GPU and multi-worker deployments.
- Obtain verified camera locations and synchronised capture clocks before evaluating cross-camera routes, and validate plate-to-vehicle association against human-reviewed sequences.
- Review authentication defaults, privacy policy, data retention and model and data licensing before any deployment beyond the coursework demonstration.

9.2 Conclusion

SAKSHI demonstrates an end-to-end AI application that combines pretrained vehicle detection and transformer-style plate OCR with camera-local tracking, conservative identity support, searchable observations and evidence-oriented storage. It also includes a web dashboard, local review rules, provenance records, privacy options and reproducible evaluation tools. The latest nine-camera profile completed 349 samples over a valid 298-second window without failed attempts, at 1.17 FPS against a 2 FPS budget. Detector-only INT8 profiling was faster than FP32 ONNX, and the plate gates rejected every mismatched reading in the small review. The result is a working system with explicit uncertainty controls, measurable operational limits and a clear path to larger deployments. A larger, authorised and independently labelled dataset is required before claiming recognition accuracy or cross-camera identity.

References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in Proc. CVPR, 2016; arXiv:1506.02640.

[2] Ultralytics, “YOLOv8 model documentation and software citation,” docs.ultralytics.com.

[3] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple Online and Realtime Tracking,” in Proc. ICIP, 2016; arXiv:1602.00763.

[4] N. Wojke, A. Bewley, and D. Paulus, “Simple Online and Realtime Tracking with a Deep Association Metric,” in Proc. ICIP, 2017; arXiv:1703.07402.

[5] R. Laroca et al., “A Robust Real-Time Automatic License Plate Recognition Based on the YOLO Detector,” in Proc. IJCNN, 2018; arXiv:1802.09567.

[6] FastALPR maintainers, “Fast-ALPR API reference,” ankandrew.github.io/fast-alpr.

[7] B. Shi, X. Bai, and C. Yao, “An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition,” IEEE TPAMI, 2017; arXiv:1507.05717.

[8] S. Mehta and M. Rastegari, “MobileViT: Light-Weight, General-Purpose, and Mobile-Friendly Vision Transformer,” in Proc. ICLR, 2022; arXiv:2110.02178.

[9] S. Mehta and M. Rastegari, “Separable Self-Attention for Mobile Vision Transformers,” TMLR, 2023; arXiv:2206.02680.

[10] B. Jacob et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in Proc. CVPR, 2018, pp. 2704-2713.

[11] E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference,” J. American Statistical Association, vol. 22, no. 158, pp. 209-212, 1927.

[12] FastPlateOCR maintainers, “Inference model zoo: model names, architecture notes and upstream results,” ankandrew.github.io/fast-plate-ocr.

[13] Open Image Models maintainers, “Plate detection model catalogue,” ankandrew.github.io/open-image-models.

[14] FastAPI, “SQL databases,” official documentation, fastapi.tiangolo.com.

[15] SAKSHI project documentation: implementation notes, evaluation register and progress log (IMPLEMENTATION.md, EVALUATION_REGISTER.md, PROGRESS.md), snapshot of 20 October 2026.

[16] SAKSHI validation artefacts: active-inference-profile-9cams-2026-10-18-300s.json, edge-benchmark-local-host-2026-10-14.json, ocr-groundtruth-evaluation-2026-10-18.json, frame-review-preparation-2026-10-18.json.

Appendix A: Reproducibility and Viva Preparation

A.1 Running the system

Install Python 3.12, Node.js 22.12 or later with npm, and `uv`, then run `uv run run.py` from the project root. The dashboard opens at `http://localhost:5174` and the API at `http://127.0.0.1:8001`. After the first installation, `uv run run.py --skip-install` restarts quickly, and `--dev-frontend` serves the UI in development mode. Six bundled clips (CAM-01 to CAM-06) run locally; CAM-07 to CAM-09 need separately supplied footage.

A.2 Questions the team should be ready to answer

Table 9.1 Viva preparation: questions and where the answer is in this report

Question	Short answer	Section
What problem does the project solve?	Turns multi-camera footage into reviewable, auditable vehicle and plate observations.	1, 2
Why an AI-based approach?	Detection and OCR need learned models; rules fail on varied scenes.	1.3
Which models were used?	YOLOv8n detector; YOLOv9-tiny plate detector; MobileViT-v2 OCR.	4.3, 4.5
Why these models?	Small, CPU-capable and available as pretrained assets.	3, 4.5
Input and output?	MP4 or RTSP video in; observations, passages, alerts and signed reports out.	4.2, 4.7
Dataset and source?	Six bundled clips plus three managed originals; no labelled test set.	6.1
Preprocessing?	Round-robin sampling, resizing to 640 and 384 inputs, privacy masking.	4.2
How was it trained and tested?	Models are pretrained; the project tests runtime, agreement and a small crop review.	6, 7
Metrics and why?	FPS, latency, CPU and memory for runtime; IoU 0.50 precision and recall planned for quality.	6.3
Main results?	1.17 FPS, zero failures, INT8 55.8 ms vs 92.8 ms, four mismatches rejected.	7
Limitations?	No held-out labels, replay timestamps, single host.	8
What next with more time or data?	Labelled dataset, OCR fine-tuning, GPU profiling.	9